

A Research on Face Liveness Detection Based on Movement Analysis and Face Features Classification by Deep Learning Model

Zhi Jie Ooi¹, Chi Wee Tan¹, Tong Ming Lim^{2,*}

¹Faculty of Computing and Information Technology, Tunku Abdul Rahman University Of Management And Technology

²Centre for Business Incubation and Entrepreneurial Ventures, Tunku Abdul Rahman University Of Management And Technology

*Corresponding author email: limtm@tarc.edu.my

Abstract: This research paper presents a study on liveness detection using face landmarking and image classification based on facial features and movements to identify individuals from a verified face database. The methodology involves detecting facial features like smiles, eye-blinks, mouth openings, and movements such as head shaking and nodding. We employed pre-trained models from two Python libraries, MediaPipe and dlib, for face landmark detection, and two self-trained Convolutional Neural Network models for detecting smiles and eye-blinks. The results demonstrate excellent performance in face landmark detection and approximately 90% accuracy in the self-trained deep learning models. These models also achieved over 90% in precision, recall, and f1-score according to the Scikit-learn classification report, thanks to the training set collected from people's face images on personal websites. Additionally, we detail the system deployment using Python's API library and an Android application prototype to validate its performance under various lighting conditions. This research offers a comprehensive analysis of the proposed method and its potential applications in real-world scenarios, such as security systems and mobile devices.

Keywords: Liveness Detection, Face Landmarking, Perspective-n-Point problem, Deep Learning, Computer Vision, Face Motion.

1. Introduction

The field of face liveness and face recognition is rapidly advancing, with a dynamic research landscape focused on developing advanced techniques to achieve precise and reliable results. Among the most promising research areas are deep learning methods that analyze vast amounts of facial images and videos using neural networks, as well as traditional geometric modeling and statistical pattern recognition.

The significance of face liveness and face recognition research lies in its potential to revolutionize a wide range of applications, including security, law enforcement, marketing, and advertising. For instance, accurate face recognition technology can greatly assist law enforcement agencies in identifying suspects and tracking their movements, while also enabling businesses to better understand their customers and target their advertising more effectively.

According to Mordor-Intelligence, an analytical firm, the global market for face recognition technology was valued at

\$4.4 billion in 2019 and is predicted to exceed \$10.9 billion by 2025. Notably, this technology has gained popularity in specific nations, such as China. The growth of the face recognition market is driven by the increasing demand for advanced security and surveillance systems across various industries, including government, healthcare, and retail.

However, ethical concerns surrounding face liveness and recognition technology, particularly regarding issues of privacy, bias, and discrimination, are also crucial. Therefore, continuous research in this area is essential to ensure that these technologies are developed and deployed responsibly and ethically.

2. Problem Statement

Face recognition technology has gained widespread adoption in contemporary society, finding applications in various consumer and industrial sectors. This technology offers a convenient and efficient means of verifying an individual's identity by comparing their face against a stored database. However, its growing prevalence has also brought to light security challenges that need to be addressed.

One of the major concerns with face recognition technology is the vulnerability to false image attacks [1]. In these attacks, malicious individuals can exploit the system by presenting fake images of an individual's face, thus gaining unauthorized access to secure areas or sensitive data without the need for additional security validation [2]. Such attacks are particularly worrisome in high-security environments, where stringent controls are in place to protect critical information and restrict access.

For instance, the banking sector relies on face recognition technology to authenticate customers during online transactions. However, the risk of a false image attack poses a significant threat, as it could enable unauthorized individuals to gain access to bank accounts and conduct fraudulent transactions.

Addressing the issue of false image attacks in face recognition technology is crucial to ensuring the security and integrity of sensitive systems. As the technology continues to evolve and integrate into various aspects of our lives, it becomes imperative to develop robust countermeasures to

safeguard against potential exploits and protect individuals and organizations from security breaches. These articles also delve into the existing vulnerabilities of the face recognition system and the potential challenges it may encounter when confronted with robust attacks employing fake images:

Table 1. A table with an example caption goes here

Article	Topic Discussed
Face recognition under spoofing attacks: countermeasures and research directions.	Discuss of methods of attack, existing methodology and given solution [3]
A comparative study on face spoofing attacks.	Need to provide more generalized algorithms for detection of unpredictable spoofing attacks [4].
Face Bio-metrics Under Spoofing Attacks: Vulnerabilities, Countermeasures, Open Issues, and Research Directions	Discussed about spoofing and anti-spoofing in face bio-metrics, and to highlight open issues and future directions. [5]
A Study on Spoofing Face Detection System.	Comprehensive overview on the research that has been carried over the last decades in the emerging field of anti-spoofing. [6]
Statistical Analysis of Image Quality Measures for Face Liveness Detection.	A technique for discriminating live from fake images given. Based upon the hypothesis that spoofing scheme leave statistical indication or structure in images. [7]

3. Objectives and Proposed Solution

The aim of this research is to investigate methods and algorithms for detecting and validating whether an individual in front of the camera is a genuine person or a spoofed image (e.g., photo or print). The proposed Face Recognition and Liveness Detection (FRLD) algorithm will focus on detecting facial features and movements using face landmarks.[8–12] This algorithm will be implemented in web applications, mobile applications, and kiosk machines. A face validation module will be employed to identify individuals through a pre-defined face image database. The study will utilize pre-trained models, such as BlazeFace and dlib 68 landmarks [13–15], for fast face landmarking, along with self-trained TensorFlow neural networks for accurate detection of smiling and eye-blinking. The algorithms designed and implemented in this research have the potential to greatly improve the security and efficiency of identity verification processes in various applications.

4. Advantages and Contributions

The research presents a compelling alternative that rivals the performance of major technology companies like Google, Amazon, and Meta for small and medium-sized enterprises (SMEs) seeking to embrace digitalization in their operations. By providing a set of algorithms and a prototype, this study offers a more cost-effective option for micro and SMEs on their digitalization journey. As a result, this alternative leads to reduced implementation costs, making it more affordable and accessible for industries in Malaysia.

Furthermore, the research outcomes contribute to accelerating the realization of IR 4.0 transformation for SMEs, as they can adopt digitalization at a lower cost and encounter fewer entry barriers. This unique solution empowers

5. Literature Review

5.1 Face recognition

Face recognition is a vital component of image processing, employing computers to identify and distinguish human faces from videos and images. Similar to how humans recognize faces by comparing them with familiar ones stored in memory, computers employ face recognition systems for this purpose. When an image or video is fed into the system, it compares the stored faces and outputs the level of similarity with the presented face. The first-ever face recognition system was developed by Bledsoe et al. in 1964, capable of detecting faces through their distinctive features. However, it necessitated a semi-automatic approach, requiring operators to input twenty metrics concerning facial attributes such as eye or mouth size. Over time, face recognition technology has undergone remarkable progress and finds extensive applications in various fields, including security, marketing, and entertainment [16].

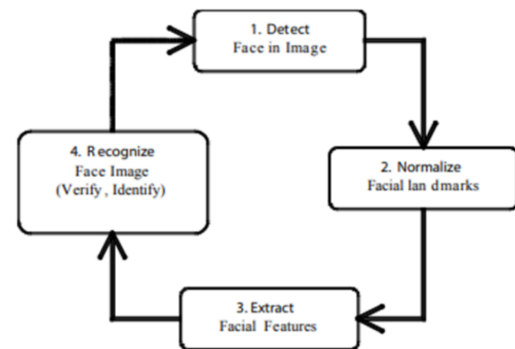


Figure 1. Steps in facial recognition algorithm [16]

Face recognition technology operates by extracting facial features from an image or video input. This process involves detecting landmarks on the face, such as the nose, mouth, and eyes. Once these landmarks are identified, they are converted into numerical values through normalization. These numerical values serve as input for a machine learning model, which is trained to recognize and categorize various faces. The machine learning model analyzes the numerical values of the facial features and compares them to a database of known

faces to ascertain the identity of the person in the input image or video.

5.2 Liveness Detection

Liveness detection holds significant importance in a face recognition system as it plays a crucial role in preventing fraud and ensuring the system's accuracy. Without liveness detection, the system could be vulnerable to manipulation using static images or videos, rather than genuine live interactions. This could lead to erroneous identifications and potentially hazardous situations [17].

5.2.1 Motion

One of the approaches to achieving liveness detection involves utilizing motion as an indicator. When capturing a person's face in real-time, the 3D shape of their face changes with movement, making it challenging to deceive the system using a static 2D image. However, employing motion as an indicator has its limitations. For instance, it necessitates high-quality video for training, which may slow down the face recognition process. Additionally, it can be deceived by 3D sculptures [17].

5.2.2 Texture

The texture indicator relies on calculating the surface of the "face" input to the face recognition system. By training the model to detect surfaces, it becomes possible to determine if the face input is a genuine image or a mere photo on paper or a screen. Using this indicator offers the advantage of convenient implementation since it only requires samples of real and non-real faces. However, a drawback of using texture as an indicator is its inability to detect surface texture when input images or videos are of low quality, and it demands more samples for texture detection training [17].

5.2.3 Life Sign

Lastly, the life sign appears to be the optimal option for implementing a faster face bio-metric authentication system. The life sign indicator necessitates user interaction with the camera to detect life signs, such as moving the face in specific directions or blinking eyes. Nevertheless, a disadvantage of this indicator is that it won't work when facial features are covered by hands or when wearing a face mask [17].

5.3 Dataset

Whitehill et al. [18] introduced the GENKI dataset, which comprises 63,000 unfiltered images gathered from personal web pages on the internet. Unlike images collected by laboratory scientists, these were captured by ordinary individuals worldwide for their personal use. The dataset encompasses three distinct categories: Smile, Non-Smile, and Unclear. For the purposes of our project, where rapid detection is required, we will focus solely on the GENKI-4K subset. This subset contains 4,000 images, specifically chosen to include only the Smile and Non-Smile categories.

6. Results

6.1 Proposed FRLD Algorithm

The FRLD algorithm proposed in this paper is a combination of Face Recognition & Liveness Detection functionalities. It comprises five distinct functions based on MediaPipe Facemesh, dlib 68 landmarks, and TensorFlow Convolutional Neural Networks. Face shaking and nodding functions rely on MediaPipe FaceMesh, which depends on the BlazeFace face detection model [15] to determine face direction. Eye-blinks and Smiling functions are processed by separate trained TensorFlow CNN models, producing Boolean outputs based on specific detection needs. The mouth opening function calculates the gap between the upper and lower lips using dlib 68 landmarks [13, 19]. FRLD enables easy implementation of five different liveness detections for real individuals in front of the camera.

The self-trained TensorFlow CNN for liveness detection uses separate training and validation sets. For instance, the smile detection CNN is trained with 20% validation set (981 images) and 80% training set (3921 images). The model's output predicts whether an input image shows a person smiling or not.

Ultimately, the project aims to implement a model capable of extracting both face and liveness features from input, enabling face recognition. The resulting process will be thoroughly evaluated to determine its success in verification.

6.1.1 Liveness Detection: Head Shake and Head Nod Module

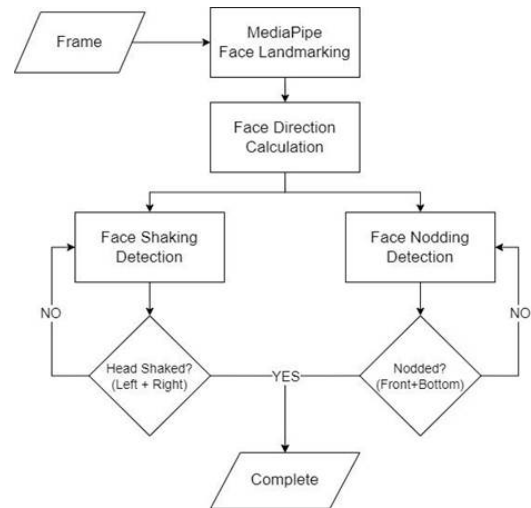


Figure 2. Flowchart of Head Shake and Head Nod Module

In the Head Shake and Head Nod Module, frames undergo facial landmark detection using MediaPipe FaceMesh [15], which relies on the BlazeFace model [13] to locate key facial landmarks in 2D (x, y) and 3D (x, y, z). After identifying the landmarks, the face direction is calculated based on that incorporating Camera Calibration and Rodrigues' rotation formula [20] to determine the nose angle in 2D. This enables the detection of face shaking or nodding.

The improved Face Orientation Direction algorithm, based

on Camera Calibration and Rodrigues' rotation formula [20] on the MediaPipe FaceMesh, is as follows:

- (i) To solve the face angle calculation using face landmarks, we refer to the Perspective-n-Point (PnP) Problem [21]. The equations for the PnP problem are as follows:

$$m' = A[R|t]M' \quad (1)$$

Or,

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (2)$$

- (u, v) are the coordinates of the projection point in pixels
 - A is a camera matrix, or a matrix of intrinsic parameters
 - (c_x, c_y) is a principal point that is usually at the image center
 - f_x, f_y are the focal lengths expressed in pixel units
 - (X, Y, Z) are the coordinates of a 3D point in the world coordinate space
 - And $[R|t]$ is the rotation and transformation matrix
- (ii) The head pose angle is derived from the PnP problem calculation. Assuming X and Y as angles, we draw a line from the middle point of the 2D graph $(0,0)$. The face direction can be determined as follows:
- $y = +ve$ and $|y| = Threshold$, then head pose should be "Look Up"
 - $y = -ve$ and $|y| = Threshold$, then head pose should be "Look Down"
 - $x = +ve$ and $|x| = Threshold$, then head pose should be "Look right"
 - $x = -ve$ and $|x| = Threshold$, then head pose should be "Look Left"
 - If four requirements above not fulfilled, consider the head pose is "Look Front"
- (iii) Refer to the 3 as direction reference

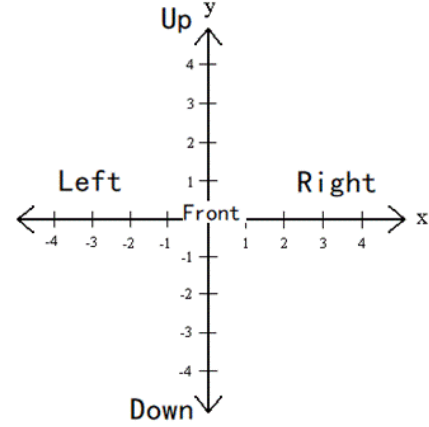


Figure 3. Direction graph reference for face direction detection

6.1.2 Liveness Detection: Eye-Blink Module

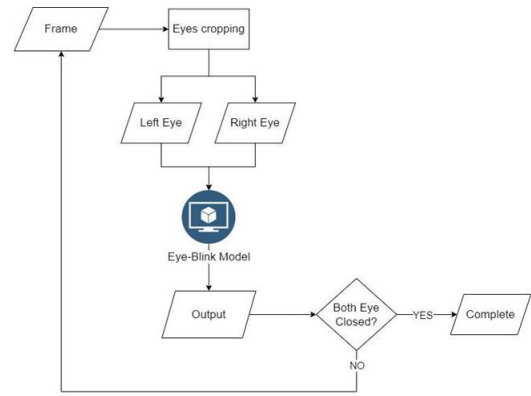


Figure 4. Flowchart of Blink Module

In the blink module, frames are initially processed through the open-cv haarcascades algorithm's left eye and right eye configurations to obtain cropped images of both eyes. These cropped images are then input into the Eye-Blink TensorFlow Model, which generates binary outputs ("0" for not closed and "1" for closed) for each eye. If both eyes are detected as closed (output: "1" for both), the module registers a blink and concludes its operation. The combined approach of open-cv haarcascades algorithm and the Eye-Blink TensorFlow Model follows these steps:

- (i) The original frame with the face is cropped to extract the left and right eyes using the open-cv haarcascades algorithm's configurations.
- (ii) The cropped eye frames are resized to a 24 x 24-pixel image, meeting the input size requirements of the model.
- (iii) This improved input process allows the Eye-Blink Module to detect human faces in any resolution, as the resolution is lowered for faster detection speed in the Eye-Blink TensorFlow Model.

The simplified binary output ("0" or "1") facilitates easy interpretation of the Boolean result for blink detection, where "0" represents no blink and "1" indicates a blink occurrence.

6.1.3 Liveness Detection: Smile Module

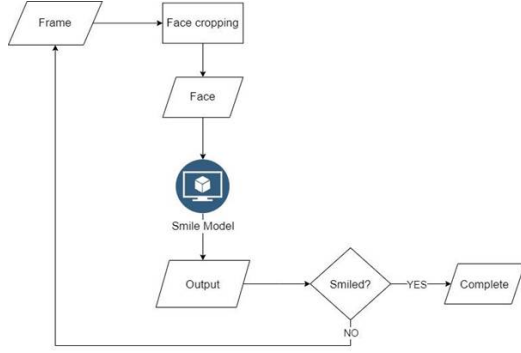


Figure 5. Flowchart of Smile Module

In the smile module, frames undergo initial processing using the open-cv haarcascades algorithm to crop the face and obtain face images. These images are then fed into the Smile TensorFlow Model, which outputs a "0" or "1" (non-smile or smile) for the face expression. If the output indicates a smile, the module concludes that a smile has occurred and completes its operation. The combined open-cv haarcascades algorithm and the Smile TensorFlow Model operate as follows:

- (i) The original frame with the face crops only the face using the open-cv haarcascades algorithm's front face configuration.
- (ii) The cropped face frame is resized to a 64 x 64-pixel image to match the model's input size.
- (iii) This input enhancement allows the Smile Module to detect human face images of any resolution since it lowers the resolution for faster detection speed in the Smile TensorFlow Model.
- (iv) The simplified output of "0" and "1" facilitates easy interpretation of the Boolean result for smile detection, with "0" representing a non-smiling face and "1" indicating a smiling face.

6.1.4 Liveness Detection: Mouth Opening Module

In the mouth opening module, the frames undergo initial processing by passing them through the dlib 68 face landmarking model, which effectively detects and locates the facial landmarks. Subsequently, the identified landmarks enable the calculation of the Mouth Aspect Ratio (MAR), represented as an integer value. The algorithm will be explain with the aid of formula below.

Upon setting an appropriate threshold value, the algorithm becomes capable of determining whether the mouth in the given image is open or closed. Specifically, when the computed MAR exceeds the allocated threshold value, the mouth is considered to be in an open state ($MAR > \text{threshold}$).

The step-by-step procedure of the enhanced algorithm is presented below:

- (i) To get the mouth landmarks, we first refer to 68 face landmarks (Fig.6) with every indexed landmark.

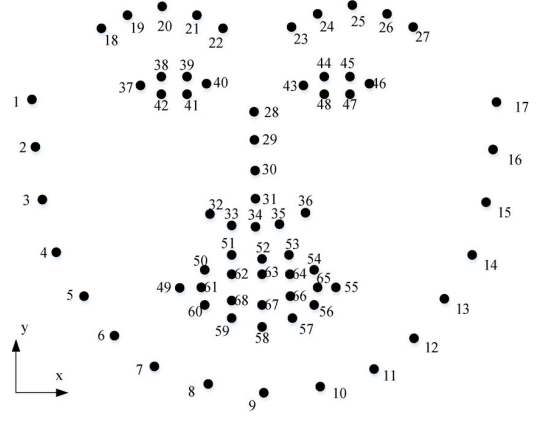


Figure 6. dlib 68 Face Landmark Reference Index [13]

- (ii) After that, the calculation of Mouth Aspect Ratio (MAR) will be following the equation,

Vertical Euclidean distances of right side of mouth,

$$A = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} \quad (3)$$

$$p(x, y) = \text{Landmark } 51, q(x, y) = \text{Landmark } 59 \quad (4)$$

Vertical Euclidean distances of left side of mouth,

$$B = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} \quad (5)$$

$$p(x, y) = \text{Landmark } 53, q(x, y) = \text{Landmark } 57 \quad (6)$$

Horizontal Euclidean distances of mouth,

$$C = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} \quad (7)$$

$$p(x, y) = \text{Landmark } 49, q(x, y) = \text{Landmark } 55 \quad (8)$$

Mouth Aspect Ratio,

$$MAR = (A + B) \div (2.0 \div C) \quad (9)$$

- (iii) When MAR is larger than threshold value, the mouth will be considered as open

6.2 Implementation of the Proposed Algorithm and Validation

6.2.1 Testing of face landmarking pre-trained model (MediaPipe & dlib)

The landmarking function of MediaPipe offers developers the flexibility to utilize it for various purposes in capturing facial features and landmarks. For instance, by employing the FaceMesh function in MediaPipe, one can easily visualize the detected facial area and landmarks. Moreover, the face landmarking function available in the dlib library also presents robust facial landmarking capabilities, as evidenced by its remarkable performance in mouth detection.



Figure 7. Result of FaceMesh using MediaPipe when proceed the Face Shaking and Face Nodding Module



Figure 8. Result of face landmarking using dlib when proceed the Mouth Opening Module

6.2.2 Testing of self-trained TensorFlow model (Smiling & Eye-Blink)

- Smile Detection

In training, the GENKI-4K and Smiling or Not datasets were used. The training scoring evaluation shows that the model can reach more than 90% of scores in terms of precision, recall, and f1-score when identifying the grin image, and the model's total accuracy is 91% when detecting the smile and non-smiling image.

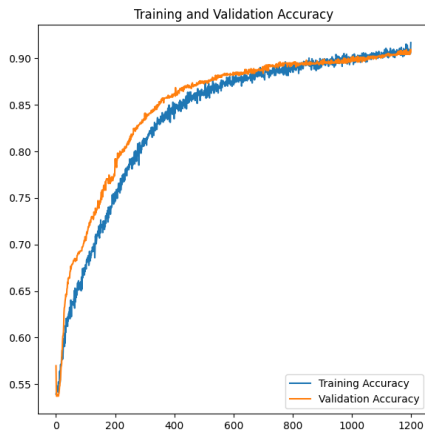


Figure 9. Training and validation accuracy of Smile Tensorflow Model

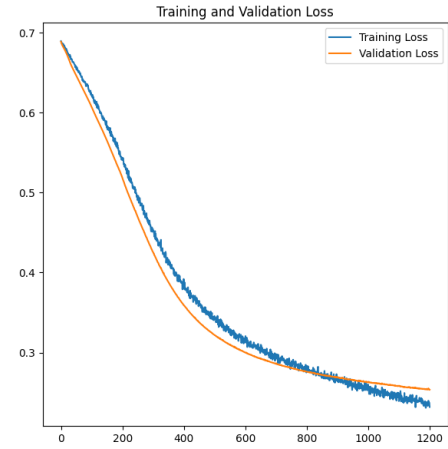


Figure 10. Training and validation loss of Smile Tensorflow Model

Table 2. Classification report of Smile Tensorflow Model

	precision	recall	f1-score	support
Non-Smile (Class 0)	0.90	0.91	0.90	454
Smile (Class 1)	0.92	0.91	0.92	527
accuracy			0.91	981
macro avg	0.91	0.91	0.91	981
weighted avg	0.91	0.91	0.91	981

- Eye-Blink Detection

The training scoring evaluation shows that the model can obtain more than 90% of scoring in terms of precision, recall, and f1-score when detecting the closed eye picture, and the model's total accuracy is 92% when identifying the open and closed eye.

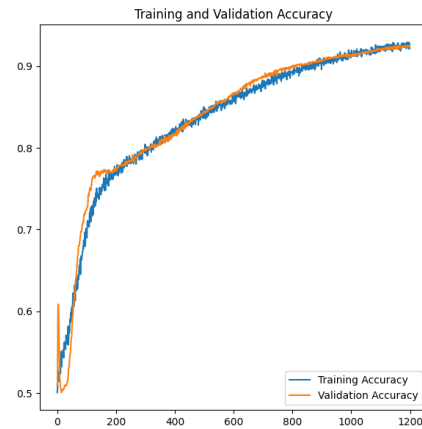


Figure 11. Training and validation accuracy of Eye-blink Tensorflow Model

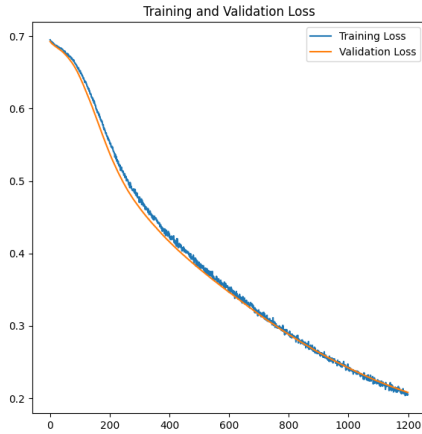


Figure 12. Training and validation loss of Eye-blink Tensorflow Model

Table 3. Classification report of Eye-Blink Tensorflow Model

	precision	recall	f1-score	support
Open (Class 0)	0.92	0.93	0.92	543
Closed (Class 1)	0.93	0.92	0.92	525
accuracy			0.92	1068
macro avg	0.92	0.92	0.92	1068
weighted avg	0.92	0.92	0.92	1068

7. Deployment

To assess the reliability of the FRLD algorithm, deployment becomes essential to identify any potential issues in the proposed detection logic.

For the API deployment, FastAPI [32] will be utilized as it allows for swift deployment of a prototype API. FastAPI is a lightweight Python-based web framework designed for creating high-performance APIs using standard Python type hints. Its use of open standards enables the generation of production-ready code and automatic interactive documentation.

On the client side, the Android platform will serve as the front-end client for capturing front camera images. These images will be transmitted to the server using the Volley library, which can send images as string data. Upon receiving the image data, the server will leverage Python's image processing libraries, such as OpenCV or Pillow, to analyze and manipulate the images as required. Subsequently, the server will respond to the Android client with the results of the image processing, which may include text extracted from the image or object recognition data.

This communication between the front-end and back-end will be facilitated through the use of RESTful API endpoints. RESTful API endpoints offer a standardized approach to transmit data between different systems, ensuring efficient and reliable data exchange.

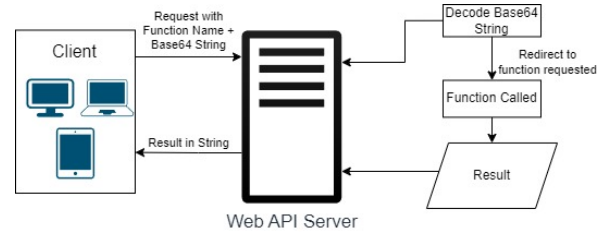


Figure 13. Visualization of Deployment Flow Between API server and Android Client

During the server setup, API testing can be conducted using any programming language capable of converting images into base64 strings and sending them through API requests to the web server. This allows for comprehensive testing of the API's functionality, ensuring that images can be accurately transmitted and processed by the server.

As for the Android implementation, the image can be captured through the device's camera and saved as a bitmap image file. The bitmap image can then be decoded into a base64 string format before being sent to the server. This process ensures that the image is transmitted securely and can be easily processed by the server upon receipt.

Utilizing base64 encoding facilitates the inclusion of binary data, such as images, within the API request. This encoding ensures that the image data is safely transmitted over networks without the risk of data corruption or loss during transmission.

Overall, the combination of base64 encoding for image data and the implementation of RESTful API endpoints creates a robust and reliable communication channel between the front-end Android client and the back-end server. This setup enables seamless testing and deployment of the FRLD algorithm in real-world scenarios, providing valuable insights into the algorithm's performance and reliability in practical applications.

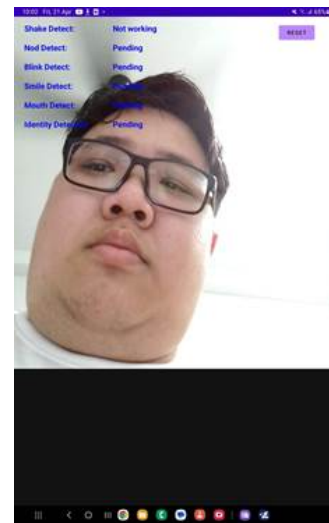


Figure 14. Interface of Android application prototype

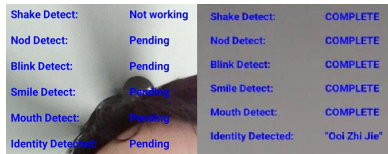


Figure 15. Information panel of Android application prototype

8. Discussion

8.1 Achievements

The achievement part of this research showcases the impressive capabilities of MediaPipe and dlib in face direction and mouth opening computation without the need for additional machine learning or image preprocessing. Although the smile detection model faces challenges due to limited training data, it still performs well overall.

The successful deployment of the Android application prototype is noteworthy as it maintains functionality even when transmitting images to the server using base64 encoding. The Web API server also operates effectively, despite encountering minor connectivity issues when accessing different internet networks, like accommodation versus university. This discrepancy is attributed to the university's firewall, which inadvertently blocks potential spamming of requests, serving as a protective measure against DDOS attacks.

8.2 Limitations and Future Improvements

Due to time constraints, this project could only implement a partially trained model combined with a pre-trained model. To enhance the project, incorporating additional datasets or exploring different Deep Learning models beyond Convolutional Neural Networks could be valuable. Evaluating transfer learning for the face features model against this project's approach would also be beneficial.

Further improvements involve fine-tuning the model with a larger dataset and employing advanced techniques like adversarial training to enhance system resilience against attacks. Additionally, the current client prototype's simplicity and bugs hinder optimal functionality. Deploying it on a website would resolve these issues, allowing easy updates without inconveniencing users. A web-based deployment would offer flexible and efficient access, handling multiple requests and providing real-time feedback. It could also allow more customization options and integration with biometric authentication or identity management platforms, enhancing overall performance.

9. Conclusion

In conclusion, this study involved the development and testing of an API prototype on an Android mobile device to assess the reliability of the API for mobile applications. The successful detection of liveness using the front camera image on a mobile phone indicates the feasibility of processing images on servers or serverless services. Additionally, this concept aligns well with the principles of edge computing [22].

Our team anticipates that this research will draw the attention of other researchers, leading to the proposal of more advanced solutions for detecting face liveness. By addressing false positives in face recognition systems, this study contributes to enhancing the accuracy of identifying individuals. This improvement holds particular significance in applications such as access control and identity verification, where false positives could lead to unauthorized access and security breaches.

Furthermore, the versatility of this project allows its implementation in various domains, including access control, identity verification, and surveillance. Industries such as hospitals, government buildings, sports arenas, and concert venues can benefit from this technology. The integration of liveness detection into these systems increases their reliability and security, providing enhanced protection and peace of mind to users.

In summary, the successful development of the API prototype and the demonstrated benefits of liveness detection showcase the potential for significant advancements in mobile applications and security systems. Further research in this area can lead to the creation of even more robust and sophisticated solutions, benefiting society as a whole.

Acknowledgment

The authors would like to thank Tunku Abdul Rahman University of Management and Technology (TAR UMT) for providing financial support and technical support when completing this study.

References

- [1] J. K. Khan and D. Upadhyay, "Security issues in face recognition," in *2014 5th International Conference - Confluence The Next Generation Information Technology Summit (Confluence)*, 2014, pp. 719–725.
- [2] R. Singh, A. Agarwal, M. Singh, S. Nagpal, and M. Vatsa, "On the robustness of face recognition algorithms against attacks and bias," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 09, p. 13583–13589, Apr 2020.
- [3] L. Li, P. L. Correia, and A. Hadid, "Face recognition under spoofing attacks: countermeasures and research directions," *IET Biometrics*, vol. 7, no. 1, pp. 3–14, 2018. [Online]. Available: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/iet-bmt.2017.0089>
- [4] S. Kumar, S. Singh, and J. Kumar, "A comparative study on face spoofing attacks," in *2017 International Conference on Computing, Communication and Automation (ICCCA)*, 2017, pp. 1104–1108.
- [5] A. Hadid, "Face biometrics under spoofing attacks: Vulnerabilities, countermeasures, open issues, and research directions," in *2014 IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2014, pp. 113–118.
- [6] P. Kavitha and K. Vijaya, "A study on spoofing face detection system," *International Journal of Pure and Applied Mathematics*, vol. 117, no. 22, pp. 205–208, 2017.
- [7] E. A. Raheem and S. M. S. Ahmad, "Statistical analysis of image quality measures for face liveness detection," in *10th International Conference on Robotics, Vision, Signal Processing and Power Applications*, M. A. M. Zawawi, S. S. Teoh, N. B. Abdullah, and M. I. S.

- Mohd Sazali, Eds. Singapore: Springer Singapore, 2019, pp. 543–549.
- [8] Y. Li, Y. Li, Q. Yan, H. Kong, and R. H. Deng, “Seeing your face is not enough: An inertial sensor-based liveness detection for face authentication,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 1558–1569. [Online]. Available: <https://doi.org/10.1145/2810103.2813612>
- [9] D. Garud and S. Agrwal, “Face liveness detection,” in *2016 International Conference on Automatic Control and Dynamic Optimization Techniques (ICACDOT)*, 2016, pp. 789–792.
- [10] J. Yang, Z. Lei, S. Liao, and S. Z. Li, “Face liveness detection with component dependent descriptor,” in *2013 International Conference on Biometrics (ICB)*, 2013, pp. 1–6.
- [11] A. Ali, F. Deravi, and S. Hoque, “Liveness detection using gaze collinearity,” in *2012 Third International Conference on Emerging Security Technologies*, 2012, pp. 62–65.
- [12] L. Wang, X. Ding, and C. Fang, “Face live detection method based on physiological motion analysis,” *Tsinghua Science and Technology*, vol. 14, no. 6, p. 685–690, Dec 2009.
- [13] M. Jabberi, A. Wali, B. B. Chaudhuri, and A. M. Alimi, “68 landmarks are efficient for 3d face alignment: what about more?” *Multimedia Tools and Applications*, Apr 2023. [Online]. Available: <https://doi.org/10.1007/s11042-023-14770-x>
- [14] S. Hangaragi, T. Singh, and N. N., “Face detection and recognition using face mesh and deep neural network,” *Procedia Computer Science*, vol. 218, pp. 741–749, 2023, international Conference on Machine Learning and Data Engineering. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050923000546>
- [15] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, and M. Grundmann, “Blazeface: Sub-millisecond neural face detection on mobile gpus,” *CoRR*, vol. abs/1907.05047, 2019. [Online]. Available: <http://arxiv.org/abs/1907.05047>
- [16] P. Shaha, U. Sharma, and K. Pawar, “Face recognition technology,” *International Journal of Research in Engineering, Science and Management*, vol. 1, no. 9, p. 149–151, Sep 2018. [Online]. Available: https://www.ijresm.com/Vol_1_2018/Vol1_Iss9_September18/IJRESM_19_32.pdf
- [17] S. Chakraborty and D. Das, “An overview of face liveness detection,” *International Journal on Information Theory*, vol. 3, no. 2, p. 11–25, Apr 2014. [Online]. Available: <https://arxiv.org/pdf/1405.2227>
- [18] J. Whitehill, G. Littlewort, I. Fasel, M. Bartlett, and J. Movellan, “Toward practical smile detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 11, p. 2106–2111, Nov 2009. [Online]. Available: <https://pdfs.semanticscholar.org/ef40/c23ec2107857eb7438ff4f9a51325bc5b046.pdf>
- [19] D. E. King, “Dlib-ml: A machine learning toolkit,” *J. Mach. Learn. Res.*, vol. 10, p. 1755–1758, dec 2009.
- [20] L. Fraiture, “A history of the description of the three-dimensional finite rotation,” *The Journal of the Astronautical Sciences*, vol. 57, no. 1, pp. 207–232, Jan 2009. [Online]. Available: <https://doi.org/10.1007/BF03321502>
- [21] F. Rocca, M. Mancas, and B. Gosselin, “Head pose estimation by perspective-n-point solution based on 2d markerless face tracking,” in *Intelligent Technologies for Interactive Entertainment*, D. Reidsma, I. Choi, and R. Bargar, Eds. Cham: Springer International Publishing, 2014, pp. 67–76.
- [22] X. Zhang, Z. Cao, and W. Dong, “Overview of edge computing in the agricultural internet of things: Key technologies, applications, challenges,” *IEEE Access*, vol. 8, pp. 141 748–141 761, 2020.